Fiches de Cours

Master IMA

Axel PIGEON
axel.pigeon@univ-tlse3.fr

Université Toulouse III – Paul Sabatier
Années universitaires 2025–2027

10 novembre 2026

Table des matières

I Apprentissage Automatique **3**

1 Fondements, Réseaux Denses et Optimisation **4**
1.1 Généralités 4
1.2 Du Perceptron au MLP 6
1.3 Fonctions d'Activation et Classification Multiclasse 8
1.4 Fonctions de Perte (*Loss Functions*) 9
1.5 Rétropropagation du Gradient et Dérivation Automatique 9

2 Réseaux Convolutifs **12**
2.1 L'Opérateur de Convolution en Vision Artificielle 12
2.2 Sous-Échantillonnage, Champ Réceptif et Calcul 13
2.3 Techniques de Normalisation et Réseaux Résiduels 14

3 Réseaux Récurrents **16**
3.1 Modélisation Séquentielle et Réseaux Récurrents (RNN) 16
3.2 Rétropropagation à travers le Temps ($BPTT$) et Pathologies 17
3.3 Cellules à Portes de Contrôle : LSTM et GRU 17
3.4 Modèles Encodeur-Décodeur et Mécanismes d'Attention 18

4 Modèles de Markov Caché **20**
4.1 Modèles de Langage et Hypothèses Markoviennes 20
4.2 Formalisme des Modèles de Markov Cachés (HMM) 22
4.3 Résolution Algorithmique des Trois Problèmes de Rabiner 23

5 Apprentissage par Renforcement **29**
5.1 Cadre général 29
5.2 Des équations de Bellman à la Programmation Dynamique 32
5.3 Apprentissage par renforcement 35

Apprentissage Automatique

Chapitre 1

Fondements, Réseaux Denses et Optimisation

Contents

1.1 Généralités 4
1.1.1 Un peu d'histoire... 4
1.1.2 Définition et terminologie 5
1.1.3 L'apprentissage 6
1.2 Du Perceptron au MLP 6
1.2.1 Concept 6
1.2.2 Apprentissage 7
1.2.3 Perceptron Multi-Couches (MLP) 7
1.3 Fonctions d'Activation et Classification Multiclasse 8
1.3.1 Fonctions d'activation usuelles 8
1.3.2 Classification multiclasse et fonction Softmax 9
1.4 Fonctions de Perte (*Loss Functions*) 9
1.4.1 Entropie croisée binaire (BCE) 9
1.5 Rétropropagation du Gradient et Dérivation Automatique 9
1.5.1 Rétropropagation sur un neurone formel 10
1.5.2 Rétropropagation dans un MLP (*Backpropagation*) 10
1.5.3 Graphes de calcul et implémentation PyTorch 10
1.5.4 Optimisation par descente de gradient avec inertie (Momentum) . . . 11

1.1 Généralités

1.1.1 Un peu d'histoire...

Nous sommes en été 1955 aux États-Unis. La naissance de l'informatique pousse les mathématiciens John McCarthy, Marvin Minsky, Nathaniel Rochester et l'ingénieur Claude Shannon à proposer un projet de recherche sur l'intelligence articielle au Dartmouth college "A proposal for the Dartmouth summer research project on artificial intelligence". Ils obtiennent ainsi 7500 $ de la fonction Rockfeller pour organiser un atelier de travail sur les machines pensantes durant deux mois. Cependant, l'intérêt pour ce domaine de recherche est encore très faible parmi la

communauté scientifique. Les faibles performances des premières machines constituent aussi un important frein au développement de prototypes.
En 1957, Frank Rosenblatt présente le premier réseau de neurones tel que nous les connaissons aujourd'hui appelés *Perceptron*. En 1986, la notion de Perceptron multicouches est explorée et en 1955 Yann LeCun et Yoshua Bengio développe le premier réseau de neurones à convolution à la base de la vision par ordinateur.
La victoire de DeepBlue sur Garry Kasparov aux échecs en 1997 démontre la puissance et le potentiel de ces nouvelles technologies. L'affluence de données disponibles ces 30 dernières années et l'évolution des capacités de calcul des ordinateurs a ouvert la porte à des systèmes de plus en plus performants. Enfin, 2015 marque l'apogée des modèles d'IA dits "classiques" avec la victoire d'AlphaGo sur les champions d'Europe et du monde.

1.1.2 Définition et terminologie

Selon Wikipédia, l'intelligence artificielle se définit comme suit :

> *"L'intelligence artificielle (IA) est l'ensemble des systèmes informatiques capables d'effectuer des tâches typiquement associées à l'intelligence, telles que l'apprentissage, le raisonnement, la résolution de problèmes, la perception ou la prise de décision."*

Le but de l'IA est donc de reproduire numériquement le mode de pensée humaine. Ces concepts ont étés, et sont toujours, à la base des réseaux de neurones actuels puisqu'ils sont contitués de neurones organisés en couches permettant de traiter l'information. C'est donc une discipline très vaste dans lequel il est facile de confondre les notions.

L'*apprentissage automatique* en est un sous-domaine qui consiste en la création d'algorithmes capables d'apprendre à partir de données précédement collectées. Il faut donc distinguer plusieurs phases dans le processus de développement d'un tel modèle :

1. La collecte des données d'entraînement.
2. L'entraînement du modèle.
3. La phase de test des performances du modèle.
4. La mise sur le marché pour une utilisation publique.

Nous traiterons ici les phases 1 et 2 de ce processus. Les autres appartiennent à des domaines différents de l'IA. Pour le développement d'un tel modèle, nous devons donc avoir accès à des **observations** consituées d'exemples ou de données collectés. Ces données sont consituées de deux éléments : les **attributs**, appelés *features*, consituent des réprésentations numériques des observations et les **étiquettes**, ou *labels*, qui associent à chaque élément une catégorie ou une valeur. L'enjeu est donc d'entraîner un modèle à distinguer les différents attributs pour chaque observation et d'être capable de répondre correctement à la question : "Étant donné une observation, à quelle catégorie appartient-elle ?". Pour cela, le jeu de données initial, appelée *dataset*, sera séparé en deux, l'un pour l'entraînement, et l'autre pour le test.

Plus formellement, pour un ensemble $E = \{(x_i, y_i) \mid i \in [\![1, N]\!]\}$ de données consitué d'observation X et de données Y, on cherche à apprendre une fonction f telle que :

$$f : \begin{cases} X \longrightarrow Y \\ x_i \longmapsto y_j = f(x_i) \end{cases}$$

et l'on souhaite que $i = j$ pour le plus de cas possibles. C'est-à-dire que notre modèle ne se trompe pas dans la prédiction y_i de la classe de x_i. En fonction de la nature de l'ensemble Y, on distinguera plusieurs types de problèmes. Si Y est un ensemble fini, on parlera de *problème de classification*, si $Y = \{vrai, faux\}$, on parlera de *prédiction* et si $Y = \mathbb{R}$, on parlera de régression.

1.1.3 L'apprentissage

L'apprentissage constitue le cœur de ce que nous allons voir ici. Cette phase permet de définir "les bonnes réponses" au modèle. On distingue ainsi différents types d'apprentissages :

- **L'apprentissage supervisé** : on dispose ici de données étiquetées et le modèle doit apprendre à distinguer les classes des observations.
- **L'apprentissage non supervisé** : ici le modèle n'a accès qu'aux observations et doit déterminer lui-même leur classes.
- **L'apprentissage par renforcement** : le modèle est ici très différent, il se représente sous la forme d'un agent devant apprendre une série d'actions en fonction de la configuration de son environnement. Pour cela, à chaque essai, on lui attribue une récompense positive ou négative.

Nous nous intéressons ici à l'apprentissage supervisé.

1.2 Du Perceptron au MLP

Le *Perceptron* peut être vu comme le premier réseau de neurones moderne, bien que sa conception date de 1956. À cette époque, Frank Rosenblatt pense à une "solution à tout" basée sur une représentation simplifiée du cerveau humain. L'idée est de recréer un ensemble de neurones connectés les uns aux autres capables d'"apprendre".

1.2.1 Concept

On se pose donc un problème à deux classes (c_-, c_+) avec m exemples :

$$\mathcal{D} = \{(x_i, y_i) \in \mathbb{R}^p \times \{-1, 1\} \mid j \in [\![1, m]\!]\}$$

Pour chaque exemple, x_i représente l'observation et y_i la classe correspondante. On veut développer un modèle capable de déterminer la classe d'une observation brute. On effectue donc une hypothèse forte qui est que les deux classes c_- et c_+ sont séparables par un hyperplan de $\mathbb{R}^p$. Le problème revient donc à construire une famille $(w_1, \dots, w_p)$ de *poids* qui définiront cet hyperplan. Formellement, on veut "apprendre" une fonction y telle que :

$$\forall i \in [\![1, m]\!], \quad y(x) = \begin{cases} 1 \text{ si } \sum_{j=1}^{p} w_j x_j > T \\ -1 \text{ sinon} \end{cases} .$$

On appellera cette fonction y, notée $\hat{y}$ la *prédiction*. Ici notre réseau de neurones a déjà pris forme. Il se représente encore plutôt sous la forme d'un neurone :

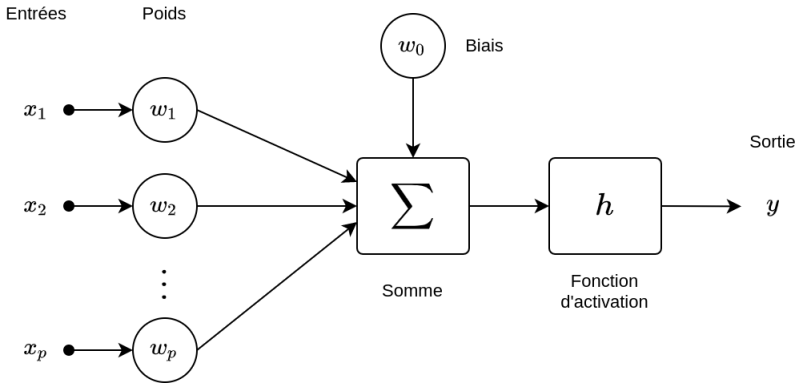


FIGURE 1.1 – Schéma formel d'un neurone

On utilisera aussi une *fonction d'activation* représentée par le seuil T. Plus tard, nous utiliserons des fonctions plus complexes pour donner une valeur de "probabilité" à la sortie d'un neurone.

1.2.2 Apprentissage

Une fois la structure posée, nous devons définir un algorithme pour apprendre les poids $(w_1, \dots, w_p)$ dans le but d'avoir une séparation optimale des deux classes dans $\mathbb{R}^p$. L'idée est donc de faire passer des valeurs x dans le perceptron et d'actualiser les poids w_j si la prédiction $\hat{y}$ est différente de la classe y. Une règle simple d'actualisation est de modifier w_j en fonction de la valeur x_j de l'entrée qu'il pondère. On aura donc la règle suivante :

$$w = \begin{cases} w + x \text{ si } y = 1 \text{ et } \hat{y} = -1 \\ w - x \text{ si } y = -1 \text{ et } \hat{y} = 1 \end{cases} \iff w = w + yx \text{ si } x \text{ est mal classé.}$$

Remarque (Variantes) Notre définition de l'actualisation des poids les modifie à chaque itération lors de l'apprentissage. Cela permet une convergence rapide de l'apprentissage, mais peut conduire à une instabilité si le nombre de poids à traiter est trop grand. Une solution est donc d'effectuer un *apprentissage par batch*. Cela consiste à faire passer successivement plusieurs vecteurs x, on calcule ensuite l'erreur moyenne commise et on met à jour les poids en fonction de cette erreur moyenne. Un autre possibilité est d'utiliser un *learning rate* ν qui bride l'apprentissage. On utilise alors la règle suivante pour actualier w :

$$w = \eta(y - \hat{y})x.$$

Les perceptrons permettent donc de simuler des fonctions booléennes comme illustré ci-dessous :

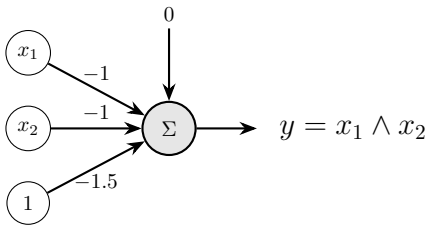


FIGURE 1.2 – Fonction AND avec un perceptron

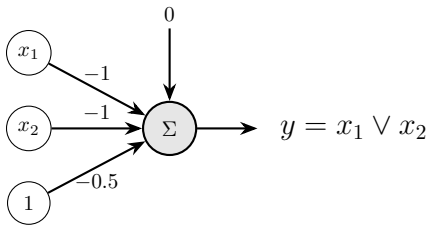


FIGURE 1.3 – Fonction OR avec un perceptron

Après réflexion, on se rend compte qu'il est impossible de simuler la fonction XOR avec un seul neurone. Ce sera donc l'objectif des *MLP (Multi Layer Perceptron)* composés de plusieurs couches de neurones interconnectés.

1.2.3 Perceptron Multi-Couches (MLP)

Comme énoncé précédemment, le perceptron simple de Rosenblatt présente une limitation fondamentale mise en lumière par Marvin Minsky et Seymour Papert en 1969 : l'impossibilité géométrique de séparer des classes qui ne sont pas linéairement séparables dans l'espace d'entrée $\mathbb{R}^d$, à l'instar de la fonction logique XOR (ou exclusif).
Pour contourner ce problème, l'approche naturelle consiste à empiler plusieurs couches de neurones formels. Un *Perceptron Multi-Couches* (Multi-Layer Perceptron, noté MLP) est composé :

1. D'une couche d'entrée (vecteur $x \in \mathbb{R}^d$) ;
2. D'une suite de $L \geqslant 1$ **couches cachées** (hidden layers), qui apprennent des représentations hiérarchiques distribuées ;
3. D'une couche de sortie produisant la prédiction $\hat{y}$.

Définition (**Formalisation de la passe avant d'un MLP**) . Soit un réseau à L couches cachées. Notons $h^{(0)} = x \in \mathbb{R}^{d_0}$ le vecteur d'entrée. Pour chaque couche $k \in [\![1, L]\!]$ de dimension d_k, le vecteur d'activation est régi par les relations de récurrence :

$$a^{(k)} = W^{(k)} h^{(k-1)} + b^{(k)} \in \mathbb{R}^{d_k} \qquad \text{(pré-activation)} \tag{1.2.1}$$

$$h^{(k)} = g^{(k)}\left(a^{(k)}\right) \in \mathbb{R}^{d_k} \qquad \text{(activation)} \tag{1.2.2}$$

où $W^{(k)} \in \mathbb{R}^{d_k \times d_{k-1}}$ désigne la matrice de poids de la couche k, $b^{(k)} \in \mathbb{R}^{d_k}$ le vecteur de biais associé, et $g^{(k)} : \mathbb{R} \to \mathbb{R}$ une fonction d'activation non linéaire appliquée composante par composante. Pour la couche de sortie $k = L + 1$, on applique une fonction terminale o :

$$\hat{y} = h^{(L+1)} = o\left(W^{(L+1)} h^{(L)} + b^{(L+1)}\right). \tag{1.2.3}$$

Remarque (Interprétation géométrique des frontières de décision) Chaque neurone d'une première couche cachée partitionne l'espace $\mathbb{R}^d$ par un demi-espace délimité par un hyperplan affine. Les neurones de la deuxième couche cachée effectuent une intersection booléenne (opérateur ET) de ces demi-espaces, formant ainsi des polyèdres convexes. Enfin, les couches suivantes peuvent réaliser l'union (opérateur OU) de régions convexes disjointes, rendant le réseau théoriquement capable d'approximer n'importe quelle frontière de décision fermée ou ouverte arbitrairement complexe.

1.3 Fonctions d'Activation et Classification Multiclasse

1.3.1 Fonctions d'activation usuelles

Le choix d'une fonction d'activation non linéaire g est indispensable : sans elle, la composition successive d'opérations affines $W^{(L)}(\dots(W^{(1)}x+b^{(1)})\dots)+b^{(L)}$ resterait strictement équivalente à une unique transformation affine globale $\tilde{W}x + \tilde{b}$, ruinant tout l'intérêt de la profondeur.

Définition (**Fonctions d'activation courantes**) . On définit ainsi différentes fonctions d'activations communément utilisées en sortie d'un neuronne. Pour tout réel $z \in \mathbb{R}$ on a :

- **Sigmoïde (logistique)** :
$$\sigma(z) = \frac{1}{1 + e^{-z}}. \tag{1.3.1}$$
Elle projette $\mathbb{R}$ sur $]0, 1[$ et vérifie l'identité remarquable $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- **Tangente hyperbolique** :
$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in]-1, 1[, \tag{1.3.2}$$
centrée en zéro, de dérivée $\tanh'(z) = 1 - \tanh^2(z)$.
- **Rectified Linear Unit (ReLU)** :
$$g(z) = \max(0, z). \tag{1.3.3}$$
Sa dérivée vaut 1 pour $z > 0$ et 0 pour $z < 0$. Elle évite la saturation des gradients pour des entrées positives.
- **Leaky ReLU** : pour $\alpha \in]0, 1[$ (couramment $\alpha = 0{,}01$), $g(z) = \max(\alpha z, z)$. Elle évite le problème des "neurones morts" lorsque $z < 0$.

- **Softplus** :
$$g(z) = \log(1 + e^z), \tag{1.3.4}$$
approximation analytique différentiable de la ReLU.

1.3.2 Classification multiclasse et fonction Softmax

Considérons un problème de classification à $K > 2$ classes mutuellement exclusives. La couche finale produit un vecteur de scores réels non normalisés $a = W^{(L+1)}h^{(L)} + b^{(L+1)} \in \mathbb{R}^K$, appelés *logits*.

Définition (**Opérateur Softmax**) . L'opérateur Softmax normalise un vecteur de logits $a = (a_1, \dots, a_K)^\top$ en une distribution discrète de probabilités $p = (p_1, \dots, p_K)^\top \in [0, 1]^K$ définie par :
$$\forall c \in [\![1, K]\!], \quad p_c = \text{Softmax}(a)_c = \frac{e^{a_c}}{\sum_{j=1}^K e^{a_j}}. \tag{1.3.5}$$
On a immédiatement $\forall c,\ p_c > 0$ et $\sum_{c=1}^K p_c = 1$. La classe prédite est $\hat{y} = \arg\max_c p_c$.

1.4 Fonctions de Perte (*Loss Functions*)

En apprentissage supervisé, puisque nous connaissons les *labels* associés à chaque prédiction, nous sommes capables d'évaluer le modèle en temps réel sur chacune de ses prédiction sur les données d'entraînement. Ainsi, l'évaluation de la discordance entre la cible réelle y et la sortie estimée $\hat{y}$ s'effectue au moyen d'une fonction de coût différentiable $\mathcal{L}(y, \hat{y})$ que l'on appellera *fonction de perte* ou *loss*.

1.4.1 Entropie croisée binaire (BCE)

Remarque (Limites de la MSE) Pour une classification binaire avec $y \in \{0, 1\}$ et une prédiction de probabilité $\hat{y} = \sigma(z) \in]0, 1[$, l'erreur quadratique usuelle MSE s'avère sous-optimale du fait de zones de saturation où le gradient s'annule pour des erreurs maximales. On introduit la perte d'entropie croisée binaire.

Définition (**Binary Cross-Entropy**) . Pour $y \in \{0, 1\}$ et $\hat{y} \in]0, 1[$:
$$\mathcal{L}_{\text{BCE}}(y, \hat{y}) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}). \tag{1.4.1}$$

Proposition (Stabilité numérique de la BCE) En injectant l'activation logistique $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$ directement dans la perte, on évite les dépassements de capacité flottante (*overflow*) via l'expression analytique exacte :
$$\mathcal{L}_{\text{BCE}}(y, \sigma(z)) = \max(z, 0) - z \cdot y + \log\left(1 + e^{-|z|}\right). \tag{1.4.2}$$

1.5 Rétropropagation du Gradient et Dérivation Automatique

Comme vu au début de ce chapitre, l'apprentissage se fait par une recherche des "meilleurs" poids possibles pour le modèle dans l'objectif de minimiser l'erreur commise. Maintenant que

nous avons introduit la fonciton de perte *loss*, nous cherchons donc à faire une minimisation de cette fonction *loss*.
La principale problématique dans ce processus est le calcul de son *gradient*. En effet, pour trouver des candidats minimiseurs, la condition d'optimalité d'ordre 1 nous demande d'annuler ce gradient. Nous devons donc être capable de calculer les dérivées partielles de la *loss* en fonction des poids du modèle. On utilise pour cela l'algorithme de *rétropropagation* à l'aide de *graphes computationnels*.

1.5.1 Rétropropagation sur un neurone formel

Considérons un unique neurone formel doté d'une entrée $x \in \mathbb{R}^d$, d'une pré-activation $a = w^\top x + b$, d'une activation $\hat{y} = \sigma(a)$, et d'une perte quadratique $\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$. Par application directe de la règle de dérivation en chaîne (*Chain Rule*) :

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{d\hat{y}}{da} \cdot \frac{\partial a}{\partial w_i} = -(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \cdot x_i. \tag{1.5.1}$$

On retrouve le terme d'erreur $(y-\hat{y})$, pondéré par la pente de la sigmoïde $\hat{y}(1-\hat{y})$ et la projection d'entrée x_i. Si l'on remplace la perte quadratique par la $\mathcal{L}_{\text{BCE}}$, le terme $\hat{y}(1 - \hat{y})$ se simplifie élégamment au dénominateur :

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial a} = \hat{y} - y \implies \frac{\partial \mathcal{L}_{\text{BCE}}}{\partial w} = (\hat{y} - y)x. \tag{1.5.2}$$

1.5.2 Rétropropagation dans un MLP (*Backpropagation*)

Dans un graphe multicouches, les erreurs locales sont propagées de l'aval vers l'amont :

Définition (**Rétropropagation multicouche**) . Posons $\delta^{(k)} = \frac{\partial \mathcal{L}}{\partial a^{(k)}}$ le vecteur de sensibilité de la couche k.

$$\text{Couche de sortie } (L+1): \quad \delta^{(L+1)} = \nabla_{a^{(L+1)}} \mathcal{L} \tag{1.5.3}$$

$$\text{Couches cachées } (k = L, \dots, 1): \quad \delta^{(k)} = \left((W^{(k+1)})^\top \delta^{(k+1)}\right) \odot g'^{(k)}\left(a^{(k)}\right) \tag{1.5.4}$$

où $\odot$ désigne le produit d'Hadamard (terme à terme). Les gradients s'en déduisent immédiatement :

$$\frac{\partial \mathcal{L}}{\partial W^{(k)}} = \delta^{(k)} \left(h^{(k-1)}\right)^\top \in \mathbb{R}^{d_k \times d_{k-1}}, \qquad \frac{\partial \mathcal{L}}{\partial b^{(k)}} = \delta^{(k)} \in \mathbb{R}^{d_k}. \tag{1.5.5}$$

1.5.3 Graphes de calcul et implémentation PyTorch

En pratique, le calcul symbolique manuel des dérivées est remplacé par l'auto-différenciation en mode inverse (*reverse-mode automatic differentiation*) matérialisée par un graphe acyclique orienté (DAG).

```python
import torch
import torch.nn as nn

class MultiLayerPerceptron(nn.Module):
    def __init__(self, in_features: int, hidden_dim: int, out_classes: int):
        super(MultiLayerPerceptron, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(in_features, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
```

```
            nn.ReLU(),
            nn.Linear(hidden_dim, out_classes)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # Retourne les logits bruts (non normalises)
        return self.network(x)

# Instanciation et passe avant/arriere
model = MultiLayerPerceptron(in_features=784, hidden_dim=128, out_classes=10)
criterion = nn.CrossEntropyLoss() # Integre log-softmax et NLL
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

x_dummy = torch.randn(32, 784) # Batch de 32 observations
y_target = torch.randint(0, 10, (32,))

optimizer.zero_grad()
logits = model(x_dummy)
loss = criterion(logits, y_target)
loss.backward() # Calcul automatique des gradients via Autograd
optimizer.step()
```

Listing 1.1 – Implémentation d'un MLP complet sous PyTorch

1.5.4 Optimisation par descente de gradient avec inertie (Momentum)

La descente de gradient stochastique standard (SGD) présente de fortes oscillations transverses dans les vallées mal conditionnées (matrice hessienne à fort conditionnement). La méthode avec moment régularise la trajectoire en accumulant une moyenne exponentielle glissante des vitesses passées :

$$v_t = \beta v_{t-1} + \alpha \nabla_W \mathcal{L}_t \quad \text{avec } \beta \in [0, 1[\tag{1.5.6}$$

$$W_{t+1} = W_t - v_t \tag{1.5.7}$$

où $\alpha > 0$ représente le pas d'apprentissage (*learning rate*).

Chapitre 2

Réseaux Convolutifs

Contents

2.1 L'Opérateur de Convolution en Vision Artificielle 12
2.1.1 Formulation mathématique 13
2.1.2 Dimensions, Stride, Padding et Dilatation 13
2.2 Sous-Échantillonnage, Champ Réceptif et Calcul 13
2.2.1 Couches de Pooling 14
2.2.2 Notion de champ réceptif (*Receptive Field*) 14
2.2.3 Algorithmique Bas Niveau : L'Approche Im2Col et GEMM 14
2.3 Techniques de Normalisation et Réseaux Résiduels 14
2.3.1 Normalisation par Lots (*Batch Normalization*) 14
2.3.2 Réseaux Résiduels (ResNet) 15
2.3.3 Implémentation d'un bloc résiduel sous PyTorch 15

Dans le cadre du traitement d'images numériques, l'utilisation de couches entièrement connectées (MLP) devient vite impraticable :

1. **Explosion du nombre de paramètres** : pour une image couleur haute définition de taille $1000 \times 1000 \times 3$, un premier neurone connecté à chaque pixel requiert 3×10^6 poids, induisant un coût mémoire prohibitif et un sur-apprentissage critique.
2. **Perte de corrélation spatiale** : vectoriser une image 2D/3D détruit la proximité géométrique locale entre pixels adjacents.
3. **Absence d'invariance par translation** : si un motif d'intérêt (un œil, une roue) se déplace dans l'image, un MLP doit réapprendre ce motif sur chacun des emplacements distincts.

La solution réside dans l'exploitation de la structure spatiale via deux principes directeurs : la *connectivité locale* (chaque neurone n'observe qu'un sous-voisinage) et le *partage des paramètres* (weight sharing).

2.1 L'Opérateur de Convolution en Vision Artificielle

L'objectif est donc d'être capable d'identifier des schémas caractéristiques dans une image, indépendamment de sa position dans celle-ci. Pour cela, on utilisera un opérateur de *convolution*. L'idée est de balayer la surface de l'image par une matrice plus petite appelée *noyau*. À chaque déplacement, on effectuera le produit de la partie de l'image couverte par le noyau. L'image ainsi obtenue sera plus petite et contiendra la même information mais sous forme condensée.

En effectuant cette étape de façon successive, on sera capable de réduire la taille de l'image d'origine à celle d'un vecteur qui pourra être facilement interprété par un MLP.

2.1.1 Formulation mathématique

En traitement du signal numérique, l'opération discrète bidimensionnelle appliquée entre une entrée matricielle I et un noyau (*kernel*) $K \in \mathbb{R}^{k_h \times k_w}$ est définie usuellement sous la forme d'une corrélation croisée :

$$S(i,j) = (I * K)(i,j) = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} I(i+m,\, j+n)\, K(m,n). \tag{2.1.1}$$

Dans une couche de convolution standard d'un CNN, l'entrée est un tenseur à trois dimensions $\mathcal{X} \in \mathbb{R}^{C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}}$ où C_{in} est le nombre de canaux (ex : Rouge, Vert, Bleu). La couche est paramétrée par C_{out} filtres tridimensionnels $W_k \in \mathbb{R}^{C_{\text{in}} \times k_h \times k_w}$ et un vecteur de biais $b \in \mathbb{R}^{C_{\text{out}}}$. La k-ième carte de caractéristiques de sortie s'exprime par :

$$S_k = b_k + \sum_{c=1}^{C_{\text{in}}} \mathcal{X}_c * W_{k,c}. \tag{2.1.2}$$

2.1.2 Dimensions, Stride, Padding et Dilatation

Propriété (Régression des dimensions spatiales) . Soit une entrée de taille $(H_{\text{in}}, W_{\text{in}})$, convoluée avec un noyau de taille (K_h, K_w), un rembourrage (*padding*) P, un pas (*stride*) S, et un taux de dilatation (*dilation*) D. La taille de la carte de sortie $(H_{\text{out}}, W_{\text{out}})$ vérifie :

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P_h - D_h(K_h - 1) - 1}{S_h} + 1 \right\rfloor, \tag{2.1.3}$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P_w - D_w(K_w - 1) - 1}{S_w} + 1 \right\rfloor. \tag{2.1.4}$$

Proposition (Nombre de paramètres d'une couche convolutive) Pour une couche convolutive associant C_{in} canaux d'entrée à C_{out} canaux de sortie au travers de noyaux $K_h \times K_w$, le nombre de paramètres scalaires libres vaut :

$$N_{\text{params}} = C_{\text{out}} \times (C_{\text{in}} \times K_h \times K_w + 1) \tag{2.1.5}$$

où le terme $+1$ comptabilise le biais scalaire associé à chaque carte de sortie. Ce nombre est strictement indépendant de la résolution spatiale $(H_{\text{in}}, W_{\text{in}})$ de l'image.

Remarque Les couches de convolutions servent donc à *extraire* des schémas (*pattern*) et les caractéristiques (*features*) de l'image et de les compresser en vecteurs. Ensuite, un MLP est chargé *d'interpréter* ces *features* en prédiction (voir chapitre précédent). L'utilisation des convolutions permet de réduire drastiquement le nombre de paramètres et de couches à utiliser pour l'extraction de ces *features*.

2.2 Sous-Échantillonnage, Champ Réceptif et Calcul

L'opération de convolution est très simple à mettre en place algorithmiquement. En fonction de l'opération effectuée avec son résultat et de la dimension de son noyau, on peut observer plusieurs propriétés remarquables. Elles permettent ensuite de définir de nouvelles couches de convolutions ayant différents cas d'usages.

2.2.1 Couches de Pooling

Les couches de regroupement (*pooling*) réduisent la résolution spatiale des tenseurs intermédiaires pour accroître l'invariance aux translations locales et diminuer la charge computationnelle :

- **Max-Pooling** : extrait la valeur maximale sur une fenêtre $p_h \times p_w$. Il conserve le signal d'activation dominant tout en rejetant les variations de fond.
- **Average-Pooling** : calcule la moyenne arithmétique sur la fenêtre, agissant comme un filtre passe-bas régularisant.

2.2.2 Notion de champ réceptif (*Receptive Field*)

Le champ réceptif d'un neurone correspond à la zone d'extension dans l'image d'entrée d'origine qui influence directement son activation.

Proposition (Croissance du champ réceptif) Pour une succession de couches convolutives à noyaux de taille k_l et de pas unitaire, le champ réceptif cumulé R_L à la couche L vérifie :

$$R_L = R_{L-1} + (k_L - 1) = 1 + \sum_{l=1}^{L} (k_l - 1). \tag{2.2.1}$$

Ainsi, l'empilement de deux convolutions successives de taille 3×3 possède un champ réceptif équivalent de 5×5, tout en utilisant $2 \times (3^2) = 18$ poids au lieu de $5^2 = 25$ pour un noyau direct, tout en introduisant deux non-linéarités intermédiaires au lieu d'une.

2.2.3 Algorithmique Bas Niveau : L'Approche Im2Col et GEMM

Les unités graphiques de calcul (GPU) n'exécutent pas les convolutions spatiales par boucles imbriquées, mais projettent l'opération sous forme de multiplication matricielle dense hautement vectorisée (*General Matrix Multiply*, GEMM) :

1. **L'algorithme Im2Col** déplie chaque bloc spatial réceptif $C_{\text{in}} \times K_h \times K_w$ présent dans le tenseur d'entrée en un vecteur colonne, formant une vaste matrice :
$$X_{\text{col}} \in \mathbb{R}^{(C_{\text{in}} \cdot K_h \cdot K_w) \times (H_{\text{out}} \cdot W_{\text{out}})}. \tag{2.2.2}$$
2. Les poids des filtres sont aplatis en lignes pour construire une matrice :
$$W_{\text{row}} \in \mathbb{R}^{C_{\text{out}} \times (C_{\text{in}} \cdot K_h \cdot K_w)}. \tag{2.2.3}$$
3. Le produit matriciel dense calcule l'ensemble des réponses en une passe :
$$Y_{\text{flat}} = W_{\text{row}} \cdot X_{\text{col}} \in \mathbb{R}^{C_{\text{out}} \times (H_{\text{out}} \cdot W_{\text{out}})}. \tag{2.2.4}$$
4. La matrice résultante est réorganisée (*reshapée*) en dimension $(C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$.

2.3 Techniques de Normalisation et Réseaux Résiduels

2.3.1 Normalisation par Lots (*Batch Normalization*)

Pour stabiliser la covariance interne des couches profondes (*Internal Covariate Shift*), on introduit la couche de Batch Normalization.

Définition (**Batch Normalization**) . Pour un mini-lot d'activations scalaires $\mathcal{B} = \{x_1, \ldots, x_m\}$ correspondant à un canal donné :

$$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i, \qquad \sigma^2_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \tag{2.3.1}$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \tag{2.3.2}$$

$$y_i = \gamma \hat{x}_i + \beta \quad \text{avec } \gamma, \beta \text{ paramètres différentiables appris.} \tag{2.3.3}$$

2.3.2 Réseaux Résiduels (ResNet)

Au-delà d'une vingtaine de couches, l'entraînement d'un réseau standard subit une dégradation critique due à l'évanescence des gradients. He et al. (2015) introduisent les connexions résiduelles directes (*skip connections*) :

$$\mathcal{H}(x) = \mathcal{F}(x, \{W_i\}) + x \tag{2.3.4}$$

où $\mathcal{F}$ représente la transformation non linéaire de quelques couches convolutives. Si la solution optimale est l'identité, le réseau n'a besoin que d'annuler les poids $\mathcal{F}(x) \to 0$, ce qui est infiniment plus aisé à apprendre. Lors de la rétropropagation, le gradient transite sans atténuation multiplicative via la dérivation directe :

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial \mathcal{H}}\left(\frac{\partial \mathcal{F}}{\partial x} + I\right). \tag{2.3.5}$$

2.3.3 Implémentation d'un bloc résiduel sous PyTorch

```python
import torch
import torch.nn as nn

class ResidualBlock(nn.Module):
    def __init__(self, channels: int):
        super(ResidualBlock, self).__init__()
        self.conv_block = nn.Sequential(
            nn.Conv2d(channels, channels, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(channels, channels, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(channels)
        )
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        identity = x
        out = self.conv_block(x)
        out += identity  # Connexion residuelle directe
        out = self.relu(out)
        return out
```

Listing 2.1 – Bloc résiduel classique sous PyTorch

Chapitre 3

Réseaux Récurrents

Contents

3.1 Modélisation Séquentielle et Réseaux Récurrents (RNN) 16
3.1.1 Architecture et Dépliement Temporel 16
3.2 Rétropropagation à travers le Temps (*BPTT*) et Pathologies . . . 17
3.2.1 Calcul du gradient BPTT 17
3.2.2 Évanescence et explosion du gradient 17
3.3 Cellules à Portes de Contrôle : LSTM et GRU 17
3.3.1 Long Short-Term Memory (LSTM) 17
3.3.2 Gated Recurrent Unit (GRU) 18
3.4 Modèles Encodeur-Décodeur et Mécanismes d'Attention 18
3.4.1 Le paradigme Sequence-to-Sequence (Seq2Seq) 18
3.4.2 Mécanisme d'attention d'alignement dynamique 18
3.4.3 Implémentation PyTorch d'un module Seq2Seq récurrent 18

3.1 Modélisation Séquentielle et Réseaux Récurrents (RNN)

Contrairement aux images dont les dimensions peuvent être ajustées à des formats fixes, les signaux temporels, acoustiques ou textuels se présentent sous la forme de séquences de longueur variable :

$$X = (x^{(1)}, x^{(2)}, \dots, x^{(T)}), \quad x^{(t)} \in \mathbb{R}^d. \tag{3.1.1}$$

3.1.1 Architecture et Dépliement Temporel

Un réseau récurrent traite séquentiellement chaque élément en maintenant un vecteur d'état interne $h^{(t)} \in \mathbb{R}^m$, constituant la mémoire synthétique du passé.

Définition (**Équations du RNN simple d'Elman**) . Pour chaque pas temporel $t \in [\![1, T]\!]$:

$$h^{(t)} = \tanh\left(Wh^{(t-1)} + Ux^{(t)} + b_h\right) \tag{3.1.2}$$

$$\hat{y}^{(t)} = \text{Softmax}\left(Vh^{(t)} + b_y\right) \tag{3.1.3}$$

où $U \in \mathbb{R}^{m \times d}$ est la matrice de projection des entrées, $W \in \mathbb{R}^{m \times m}$ la matrice de récurrence

état-à-état, et $V \in \mathbb{R}^{K \times m}$ la matrice de décodage vers l'espace de sortie. Les paramètres $\{U, W, V, b_h, b_y\}$ sont **partagés** à travers tous les pas de temps.

3.2 Rétropropagation à travers le Temps (*BPTT*) et Pathologies

3.2.1 Calcul du gradient BPTT

L'erreur globale commise sur la séquence est la somme des coûts temporels $E = \sum_{t=1}^{T} E_t$. Pour dériver le gradient par rapport à la matrice de récurrence W, on déroule la dépendance temporelle :

$$\frac{\partial E}{\partial W} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial E_t}{\partial h^{(t)}} \cdot \left(\prod_{j=k+1}^{t} \frac{\partial h^{(j)}}{\partial h^{(j-1)}} \right) \cdot \frac{\partial h^{(k)}}{\partial W}. \tag{3.2.1}$$

3.2.2 Évanescence et explosion du gradient

Le produit matriciel cumulé $\prod_{j=k+1}^{t} \frac{\partial h^{(j)}}{\partial h^{(j-1)}}$ gouverne la propagation temporelle du signal d'erreur. Si la matrice jacobienne présente des valeurs singulières strictement supérieures à 1, la norme du gradient diverge exponentiellement vers l'infini pour de grandes séquences : c'est l'**explosion du gradient**. On y remédie par la troncature heuristique (*Gradient Clipping*) :

$$g \leftarrow g \cdot \min\left(1, \ \frac{\text{seuil}}{\|g\|_2}\right). \tag{3.2.2}$$

Inversement, si les valeurs propres sont inférieures à 1, le gradient s'évanouit exponentiellement vers zéro au fur et à mesure que $t - k$ croît (**dissipation du gradient**), interdisant l'apprentissage de corrélations à long terme.

3.3 Cellules à Portes de Contrôle : LSTM et GRU

3.3.1 Long Short-Term Memory (LSTM)

Conçue par Hochreiter et Schmidhuber (1997), l'architecture LSTM résout l'évanescence du gradient via un état cellulaire interne C_t mis à jour par des opérations additives protégées par des portes logiques différentiables.

Définition (**Formulation complète d'un bloc LSTM**) . Soit x_t l'entrée au temps t et h_{t-1} l'état caché antérieur. Les portes sont régies par :

$$f_t = \sigma\left(W_f [h_{t-1}, x_t] + b_f\right) \quad \text{(porte d'oubli / Forget Gate)} \tag{3.3.1}$$

$$i_t = \sigma\left(W_i [h_{t-1}, x_t] + b_i\right) \quad \text{(porte d'entrée / Input Gate)} \tag{3.3.2}$$

$$\tilde{C}_t = \tanh\left(W_C [h_{t-1}, x_t] + b_C\right) \quad \text{(état candidat)} \tag{3.3.3}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad \textbf{(mise à jour cellulaire additive)} \tag{3.3.4}$$

$$o_t = \sigma\left(W_o [h_{t-1}, x_t] + b_o\right) \quad \text{(porte de sortie / Output Gate)} \tag{3.3.5}$$

$$h_t = o_t \odot \tanh(C_t) \quad \text{(état caché final).} \tag{3.3.6}$$

Grâce à la relation linéaire $C_t = f_t \odot C_{t-1} + \ldots$, si la porte d'oubli est activée à saturation ($f_t \approx 1$), le gradient d'erreur se propage à travers l'axe temporel sans aucune atténuation multiplicative exponentielle.

3.3.2 Gated Recurrent Unit (GRU)

Proposé par Cho et al. (2014), le bloc GRU fusionne l'état cellulaire et l'état caché tout en réduisant le nombre de portes à deux :

$$z_t = \sigma\left(W_z[h_{t-1}, x_t] + b_z\right) \quad \text{(porte de mise à jour / Update Gate)} \tag{3.3.7}$$

$$r_t = \sigma\left(W_r[h_{t-1}, x_t] + b_r\right) \quad \text{(porte de réinitialisation / Reset Gate)} \tag{3.3.8}$$

$$\tilde{h}_t = \tanh\left(W[r_t \odot h_{t-1}, x_t] + b\right) \tag{3.3.9}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \tag{3.3.10}$$

3.4 Modèles Encodeur-Décodeur et Mécanismes d'Attention

3.4.1 Le paradigme Sequence-to-Sequence (Seq2Seq)

Dans une tâche de traduction ou de résumé, la longueur de la séquence générée est distincte de la séquence source. L'encodeur compresse l'intégralité de la séquence d'entrée en un vecteur contexte unique $c = h^{(T_{\text{in}})}$. Le décodeur génère les sorties mot par mot en conditionnant chaque tirage sur ce vecteur. Ce goulot d'étranglement unique entraîne une forte détérioration des performances lorsque la phrase dépasse une vingtaine de mots.

3.4.2 Mécanisme d'attention d'alignement dynamique

L'attention (Bahdanau et al., 2015 ; Luong et al., 2015) lève cette contrainte en permettant au décodeur de consulter l'ensemble des états intermédiaires de l'encodeur $(s_1, \ldots, s_S)$ à chaque pas de génération t.

Définition (**Calcul du vecteur de contexte par attention**) . Pour un état courant du décodeur h_t et un état source s_k :

$$\text{Score d'alignement :} \quad e_{t,k} = \begin{cases} v_a^\top \tanh\left(W_a h_t + U_a s_k\right) & \text{(Attention additive de Bahdanau)} \\ h_t^\top W_a s_k & \text{(Attention générale de Luong)} \\ h_t^\top s_k & \text{(Attention par produit scalaire direct)} \end{cases} \tag{3.4.1}$$

$$\text{Poids d'attention :} \quad \alpha_{t,k} = \frac{\exp(e_{t,k})}{\sum_{j=1}^{S} \exp(e_{t,j})} \quad \in [0, 1] \tag{3.4.2}$$

$$\text{Vecteur de contexte :} \quad c_t = \sum_{k=1}^{S} \alpha_{t,k} s_k. \tag{3.4.3}$$

3.4.3 Implémentation PyTorch d'un module Seq2Seq récurrent

```python
import torch
import torch.nn as nn

class Seq2SeqRNN(nn.Module):
    def __init__(self, vocab_size: int, embed_dim: int, hidden_dim: int):
        super(Seq2SeqRNN, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.encoder = nn.LSTM(embed_dim, hidden_dim, batch_first=True,
    bidirectional=True)
```

```
        # Decodeur prenant en compte la bidirectionnalite (hidden_dim * 2)
        self.decoder = nn.LSTM(embed_dim, hidden_dim * 2, batch_first=True)
        self.fc_out = nn.Linear(hidden_dim * 2, vocab_size)

    def forward(self, src: torch.Tensor, trg: torch.Tensor) -> torch.Tensor:
        # Encodage
        embedded_src = self.embedding(src)
        enc_outputs, (h_n, c_n) = self.encoder(embedded_src)

        # Concatener les etats terminaux des deux directions de l'encodeur
        h_dec = torch.cat((h_n[0:1], h_n[1:2]), dim=2)
        c_dec = torch.cat((c_n[0:1], c_n[1:2]), dim=2)

        # Decodage autoregressif
        embedded_trg = self.embedding(trg)
        dec_outputs, _ = self.decoder(embedded_trg, (h_dec, c_dec))
        logits = self.fc_out(dec_outputs)
        return logits
```

Listing 3.1 – Encodeur LSTM et Décodeur avec projection sous PyTorch

Chapitre 4

Modèles de Markov Caché

Contents

4.1 Modèles de Langage et Hypothèses Markoviennes 20
4.1.1 Modélisation séquentielle par règle de la chaîne 20
4.1.2 Du Sac de Mots au Modèle de Markov d'ordre m 21
4.1.3 Modèles à classes de Brown 22
4.2 Formalisme des Modèles de Markov Cachés (HMM) 22
4.2.1 Définition mathématique 22
4.2.2 Les trois problèmes fondamentaux de Rabiner (1989) 23
4.3 Résolution Algorithmique des Trois Problèmes de Rabiner 23
4.3.1 Problème 1 : Évaluation et Algorithme Forward-Backward 23
4.3.2 Problème 2 : Décodage et Inférence 25
4.3.3 Stabilité Numérique et Domaine Logarithmique 26
4.3.4 Problème 3 : Apprentissage des Paramètres 27

Ce chapitre traite des *Modèles de Markov Cachés* (Hidden Markov Models, HMM). Historiquement développés pour le traitement automatique de la parole (*Automatic Speech Recognition*, Rabiner, 1989) et le traitement du langage naturel (étiquetage morpho-syntaxique *POS tagging*, modèles de langage, alignement bilingue), les HMM offrent un cadre probabiliste rigoureux pour les séquences discrètes. Bien que les modèles de fondation neuronaux (Transformers) dominent aujourd'hui ces applications, les algorithmes sous-jacents aux HMM (Forward-Backward, Viterbi, Baum-Welch/EM) constituent les bases fondamentales des modèles graphiques probabilistes et de la programmation dynamique.

4.1 Modèles de Langage et Hypothèses Markoviennes

Dans ce cours, on introduit les HMM en tant que modèles de langage. On comment ici par formaliser mathématiquement ces modèles en utilisant les chaînes de Markov.

4.1.1 Modélisation séquentielle par règle de la chaîne

Soit $\mathcal{V}$ un vocabulaire fini de symboles (mots ou *tokens*). On cherche à définir une loi de probabilité conjointe sur des séquences de longueur arbitraire bordées par deux balises déterministes start et stop :

$$O = (\text{start}, O_1, O_2, \ldots, O_n, \text{stop}) \in \mathcal{V}^*. \tag{4.1.1}$$

Par application exacte de la règle de décomposition en chaîne des probabilités conditionnelles, toute distribution sur une séquence s'écrit de manière causale (contexte gauche) :

$$\mathbb{P}(\text{start}, O_1, \ldots, O_n, \text{stop}) = \prod_{i=1}^{n+1} \mathbb{P}(O_i \mid \text{start}, O_1, \ldots, O_{i-1}) \tag{4.1.2}$$

en posant conventionnellement $O_{n+1} = \text{stop}$.

Remarque (Intuition des dés de Noah Smith et explosion combinatoire) On peut interpréter ce processus génératif à l'aide d'une collection de dés dont les faces représentent les mots de $\mathcal{V}$. À chaque étape, l'historique complet $h = (\text{start}, O_1, \ldots, O_{i-1})$ détermine le dé à lancer, dont la distribution sur les faces fournit le mot suivant O_i. Bien que d'une expressivité totale, ce modèle présente deux écueils rédhibitoires :

1. **Explosion paramétrique** : pour un historique de longueur au plus n et un vocabulaire de taille $V = |\mathcal{V}|$, le nombre d'historiques possibles vaut $\sum_{k=0}^{n-1} V^k = \frac{V^n - 1}{V - 1}$. Pour $V = 10^5$ et $n = 10$, cela exigerait de stocker les paramètres d'environ 10^{50} distributions catégorielles, ce qui est matériellement impossible.
2. **Défaut de généralisation** : toute séquence contenant un préfixe non observé lors de l'apprentissage se voit affecter une probabilité nulle.

4.1.2 Du Sac de Mots au Modèle de Markov d'ordre m

Pour rendre l'estimation calculable, on réduit l'historique conditionnel via des hypothèses d'indépendance conditionnelle.

Définition (**Modèle Sac de Mots / Unigramme**) . L'hypothèse la plus forte pose l'indépendance mutuelle totale de chaque observation vis-à-vis du contexte passé :

$$\mathbb{P}(\text{start}, O_1, \ldots, O_n, \text{stop}) = \prod_{i=1}^{n} \mathbb{P}(O_i). \tag{4.1.3}$$

Ce modèle ne requiert que $V - 1$ paramètres libres. En revanche, il détruit toute syntaxe : la probabilité d'une séquence est invariante par permutation de ses termes.

Le compromis canonique consiste à borner la mémoire du processus aux m derniers symboles émis.

Définition (**Chaîne de Markov d'ordre m / n-grammes**) . Un modèle de Markov stationnaire d'ordre $m \geqslant 1$ pose que la loi conditionnelle du token courant ne dépend que des m tokens immédiatement antérieurs :

$$\mathbb{P}(O_i \mid \text{start}, O_1, \ldots, O_{i-1}) = \mathbb{P}(O_i \mid O_{i-m}, \ldots, O_{i-1}). \tag{4.1.4}$$

La probabilité conjointe de la séquence s'écrit alors :

$$\mathbb{P}(\text{start}, O_1, \ldots, O_n, \text{stop}) = \prod_{i=1}^{n+1} \mathbb{P}(O_i \mid O_{i-m}^{i-1}) \tag{4.1.5}$$

où $O_{i-m}^{i-1} = (O_{i-m}, \ldots, O_{i-1})$.

Remarque (Taxonomie et complexité des n-grammes) Dans la terminologie du traitement automatique du langage :

- $m = 0$ correspond au modèle *unigramme* ($V - 1$ paramètres) ;
- $m = 1$ correspond au modèle *bigramme* ($V(V - 1)$ paramètres de transition) ;
- $m = 2$ correspond au modèle *trigramme* ($V^2(V - 1)$ paramètres).

Au-delà de $m = 3$ ou 4, la matrice des comptes devient extrêmement creuse (*sparsity*), rendant indispensables des techniques de lissage statistique (Good-Turing, Kneser-Ney).

4.1.3 Modèles à classes de Brown

Pour injecter de la régularité syntaxique sans subir l'explosion combinatoire des n-grammes lexicaux, Brown et al. (1992) partitionnent le vocabulaire $\mathcal{V}$ en $C \ll V$ classes syntaxiques disjointes via une application déterministe $\mathrm{cl} : \mathcal{V} \to [\![1, C]\!]$ obtenue par partitionnement statistique :

Définition (**Modèle de séquence fondé sur des classes**) . La génération d'une observation O_i est scindée en deux lois : une transition d'état à état entre classes, suivie de l'émission du mot conditionnellement à sa propre classe :

$$\mathbb{P}(\mathrm{start}, O_1, \ldots, O_n, \mathrm{stop}) = \prod_{i=1}^{n+1} \mathbb{P}(O_i \mid \mathrm{cl}(O_i)) \cdot \mathbb{P}(\mathrm{cl}(O_i) \mid \mathrm{cl}(O_{i-1})). \tag{4.1.6}$$

En notant $A \in \mathbb{R}^{C \times C}$ la matrice de transition inter-classes et $B \in \mathbb{R}^{C \times V}$ la matrice d'émission des mots sachant la classe, ce modèle ne requiert que $C^2 + C \times V$ paramètres.
Cependant, ce modèle suppose que la fonction de projection cl est déterministe et connue à l'avance. Or, dans la langue naturelle, un mot est fondamentalement polysémique : le mot « *flies* » peut être un verbe ou un nom selon le contexte. L'état syntaxique sous-jacent est donc par essence *non observé* : c'est l'acte de naissance des *Modèles de Markov Cachés*.

4.2 Formalisme des Modèles de Markov Cachés (HMM)

4.2.1 Définition mathématique

Un modèle de Markov caché de premier ordre modélise un processus bivarié $(S_t, O_t)_{t \geq 1}$ où la suite des états (S_t) forme une chaîne de Markov non observable (latente), et (O_t) est la suite des observations visibles émises à chaque instant conditionnellement à l'état courant.

Définition (**Paramètres canoniques d'un HMM**) . Un HMM à états discrets est formellement caractérisé par le triplet $\lambda = (\pi, A, B)$:

1. Un ensemble fini d'états cachés $\mathcal{S} = \{s_1, \ldots, s_N\}$.
2. Un alphabet fini d'observations $\mathcal{V} = \{v_1, \ldots, v_M\}$.
3. La distribution initiale des états $\pi = (\pi_i)_{i=1}^N$ avec $\pi_i = \mathbb{P}(S_1 = s_i)$.
4. La matrice de transition $A = (a_{ij}) \in \mathbb{R}^{N \times N}$ régie par l'hypothèse markovienne :

$$a_{ij} = \mathbb{P}(S_{t+1} = s_j \mid S_t = s_i), \quad \sum_{j=1}^{N} a_{ij} = 1. \tag{4.2.1}$$

5. La matrice d'émission $B = (b_j(k)) \in \mathbb{R}^{N \times M}$ vérifiant l'hypothèse d'indépendance

conditionnelle :

$$b_j(k) = \mathbb{P}(O_t = v_k \mid S_t = s_j), \quad \sum_{k=1}^{M} b_j(k) = 1. \tag{4.2.2}$$

Pour une séquence d'états $S = (S_1, \ldots, S_T)$ et d'observations $O = (O_1, \ldots, O_T)$, la loi conjointe se factorise immédiatement grâce aux indépendances conditionnelles du graphe :

$$\mathbb{P}(O, S \mid \lambda) = \pi_{S_1} b_{S_1}(O_1) \prod_{t=2}^{T} a_{S_{t-1}, S_t} b_{S_t}(O_t). \tag{4.2.3}$$

4.2.2 Les trois problèmes fondamentaux de Rabiner (1989)

L'utilisation des modèles HMM induit trois problèmes fondamentaux appelés des *problèmes de Rabiner*, en l'honneur de la personne qui les a formalisés la première fois en 1989.

Proposition (Problèmes de Rabiner - 1989) Les 3 problèmes présentés par Rabiner sont :

1. **L'Évaluation** : À partir d'une séquence d'observation donnée $O = (O_1, \ldots, O_m)$ comment calculer la probabilité qu'elle soit générée par le modèle ? L'approche naïve serait de dénombrer toutes les séquences possibles de probabilité non nulle, mais c'est un problème de complexité exponentielle par rapport à la taille de la séquence. En pratique, on utilisera l'algorithme de programmation dynnamique *Forward*.
2. **Inférence/Décodage** : À partir d'une observation $O = (O_1, \ldots, O_n)$ quelle est la séquence d'états cachés $S = (S_1, \ldots, S_n)$ qui aurait généré ces observations la plus probable ? On pourrait aussi calculer des fréquences sur toutes les séquences d'états possibles pouvant générer cette observation mais c'est aussi de complexité exponentiell. On utilisera l'algorithme de *Viterbi*.
3. **L'Apprentissage** : À partir d'un jeu de données d'observations, comment estimer les matrices de transition A et B pour construire le modèle ? On distingue ici deux cas :
   - **Apprentissage Supervisé** : On dispose ici des séquences d'observations et les états cachés correspondant : il suffit alors de calculer des fréquences dessus pour estimer les deux matrices.
   - **Apprentissage Non-Supervisé** : On ne dispose que des séquences d'observations. On utilisera pour cela l'algorithme *Expectation-Maximization (EM)* développé par Dempster en 1977. Cet algorithme se base sur le calcul d'un maximum de vraissemblance sur les états possibles.

     On peut aussi citer des approches par clustering sur les observations pour définir des états cachés ou des k-means.

4.3 Résolution Algorithmique des Trois Problèmes de Rabiner

4.3.1 Problème 1 : Évaluation et Algorithme Forward-Backward

Soit un modèle HMM fixé $\lambda = (\pi, A, B)$ et une séquence d'observations $O = (O_1, O_2, \ldots, O_T) \in \mathcal{V}^T$. Le problème d'évaluation consiste à déterminer la vraisemblance marginale $\mathbb{P}(O \mid \lambda)$. Commençons par étudier la complexité de l'approche naïve pour se convaincre de l'utilisation d'algorithmes plus performants.

Proposition (Approche naïve) Par la loi des probabilités totales, la vraisemblance marginale s'obtient en sommant la loi conjointe sur l'ensemble $\mathcal{S}^T$ des trajectoires d'états latents possibles :

$$\mathbb{P}(O \mid \lambda) = \sum_{S \in \mathcal{S}^T} \mathbb{P}(O, S \mid \lambda) = \sum_{s_{i_1}=1}^{N} \cdots \sum_{s_{i_T}=1}^{N} \pi_{s_{i_1}} b_{s_{i_1}}(O_1) \prod_{t=2}^{T} a_{s_{i_{t-1}}, s_{i_t}} b_{s_{i_t}}(O_t). \tag{4.3.1}$$

L'espace $\mathcal{S}^T$ possède un cardinal de N^T. Chaque terme nécessitant $2T-1$ multiplications, la complexité temporelle brute est en $\mathcal{O}(T \cdot N^T)$. Ce calcul devient physiquement irréalisable dès que T dépasse quelques dizaines d'instants (pour $N = 10$ et $T = 50$, $N^T = 10^{50}$).

On préférera donc l'algorithme Forward ou Backward définis ci-dessous. L'algorithme Forward résout ce problème par *programmation dynamique* en factorisant les calculs sur un treillis d'états (*trellis*).

Proposition (Algorithme Forward) On introduit la variable intermédiaire $\alpha_t(i)$, définie comme la probabilité conjointe du préfixe d'observations jusqu'à l'instant t et de l'état latent au temps t :

$$\alpha_t(i) = \mathbb{P}(O_1, \ldots, O_t, S_t = s_i \mid \lambda), \quad \forall i \in [\![1, N]\!],\, t \in [\![1, T]\!]. \tag{4.3.2}$$

Algorithm 1 Algorithme Forward

1. **Initialisation** ($t = 1$) :
$$\alpha_1(i) = \pi_i \, b_i(O_1), \quad \forall i \in [\![1, N]\!]. \tag{4.3.3}$$
2. **Induction / Récurrence** ($t = 1, \ldots, T-1$) :
$$\alpha_{t+1}(j) = \left[\sum_{i=1}^{N} \alpha_t(i) \, a_{ij} \right] b_j(O_{t+1}), \quad \forall j \in [\![1, N]\!]. \tag{4.3.4}$$
3. **Terminaison** :
$$\mathbb{P}(O \mid \lambda) = \sum_{i=1}^{N} \alpha_T(i). \tag{4.3.5}$$

Remarque (Complexité) Le treillis compte $N \times T$ nœuds. À chaque transition temporelle $t \to t{+}1$, le calcul de $\alpha_{t+1}(j)$ agrège les contributions des N états antérieurs via N multiplications et additions. La complexité temporelle chute ainsi à $\mathcal{O}(N^2 T)$, avec une complexité spatiale en mémoire de $\mathcal{O}(NT)$ (ou $\mathcal{O}(N)$ si l'on ne conserve que le vecteur de l'instant précédent).

Proposition (Algorithme Backward) De manière duale, on définit la variable Backward $\beta_t(i)$ comme la probabilité conditionnelle du suffixe d'observations allant de $t+1$ à T, conditionnellement à l'état latent présent $S_t = s_i$:

$$\beta_t(i) = \mathbb{P}(O_{t+1}, \ldots, O_T \mid S_t = s_i, \lambda), \quad \forall i \in [\![1, N]\!],\, t \in [\![1, T]\!]. \tag{4.3.6}$$

Algorithm 2 Algorithme Backward

1. **Initialisation** $(t = T)$:
$$\beta_T(i) = 1, \quad \forall i \in [\![1, N]\!]. \tag{4.3.7}$$
2. **Induction régressive** $(t = T-1, \ldots, 1)$:
$$\beta_t(i) = \sum_{j=1}^{N} a_{ij}\, b_j(O_{t+1})\, \beta_{t+1}(j), \quad \forall i \in [\![1, N]\!]. \tag{4.3.8}$$
3. **Terminaison** :
$$\mathbb{P}(O \mid \lambda) = \sum_{i=1}^{N} \pi_i\, b_i(O_1)\, \beta_1(i). \tag{4.3.9}$$

4.3.2 Problème 2 : Décodage et Inférence

Étant donnée une séquence observée O, le décodage consiste à inférer la séquence d'états latents sous-jacente. Selon l'objectif visé, deux formulations distinctes sont envisagées. Le critère le plus classique recherche la trajectoire complète $S^* = (S_1^*, \ldots, S_T^*)$ qui maximise la loi conjointe (et donc la loi a posteriori globale $\mathbb{P}(S \mid O, \lambda)$) :
$$S^* = \arg\max_{S \in \mathcal{S}^T} \mathbb{P}(S \mid O, \lambda) = \arg\max_{S \in \mathcal{S}^T} \mathbb{P}(O, S \mid \lambda). \tag{4.3.10}$$
On introduit la variable de Viterbi $\delta_t(i)$, représentant la probabilité maximale d'une séquence partielle d'états se terminant en s_i au temps t, ainsi qu'une matrice de pointeurs $\psi_t(i)$ (*back-pointers*) stockant l'antécédent optimal :
$$\delta_t(i) \triangleq \max_{S_1, \ldots, S_{t-1}} \mathbb{P}(O_1, \ldots, O_t, S_1, \ldots, S_{t-1}, S_t = s_i \mid \lambda). \tag{4.3.11}$$

Algorithm 3 Algorithme de Viterbi

1. **Initialisation** $(t = 1)$:
$$\delta_1(i) = \pi_i\, b_i(O_1), \quad \forall i \in [\![1, N]\!], \quad \psi_1(i) = 0. \tag{4.3.12}$$
2. **Récurrence** $(t = 2, \ldots, T)$:
$$\delta_t(j) = \left[\max_{1 \le i \le N} \big(\delta_{t-1}(i)\, a_{ij}\big)\right] b_j(O_t), \quad \forall j \in [\![1, N]\!], \tag{4.3.13}$$
$$\psi_t(j) = \arg\max_{1 \le i \le N} \big(\delta_{t-1}(i)\, a_{ij}\big), \quad \forall j \in [\![1, N]\!]. \tag{4.3.14}$$
3. **Terminaison** :
$$P^* = \max_{1 \le i \le N} \delta_T(i), \tag{4.3.15}$$
$$S_T^* = \arg\max_{1 \le i \le N} \delta_T(i). \tag{4.3.16}$$
4. **Reconstruction du chemin (*Backtracking*)** $(t = T-1, \ldots, 1)$:
$$S_t^* = \psi_{t+1}(S_{t+1}^*). \tag{4.3.17}$$

Proposition (*Posterior Decoding*) Une alternative consiste à déterminer, pour chaque pas de temps t pris isolément, l'état qui maximise la probabilité a posteriori conditionnelle $\mathbb{P}(S_t = s_i \mid O, \lambda)$:

$$S_*^t = \arg\max_{1 \le i \le N} \mathbb{P}(S_t = s_i \mid O, \lambda). \tag{4.3.18}$$

Grâce aux indépendances conditionnelles du modèle, la probabilité conjointe de l'état $S_t = s_i$ et de l'ensemble de la séquence d'observations O se factorise par le produit des variables Forward et Backward :

$$\mathbb{P}(S_t = s_i, O \mid \lambda) = \alpha_t(i)\, \beta_t(i). \tag{4.3.19}$$

Par la formule de Bayes :

$$\mathbb{P}(S_t = s_i \mid O, \lambda) = \frac{\mathbb{P}(S_t = s_i, O \mid \lambda)}{\mathbb{P}(O \mid \lambda)} = \frac{\alpha_t(i)\, \beta_t(i)}{\sum_{j=1}^{N} \alpha_t(j)\, \beta_t(j)}. \tag{4.3.20}$$

Le décodage a posteriori revient donc à évaluer :

$$S_*^t = \arg\max_{1 \le i \le N} \big(\alpha_t(i)\, \beta_t(i)\big). \tag{4.3.21}$$

Remarque (Comparaison Viterbi vs Décodage a posteriori)

- **Viterbi** garantit la validité structurelle globale du chemin prédit : aucune transition de probabilité nulle ($a_{ij} = 0$) ne peut être empruntée.
- **Le décodage a posteriori** minimise le taux d'erreur moyen par position temporelle. Cependant, en maximisant chaque état indépendamment de ses voisins, il peut produire deux états consécutifs S_*^t et S_*^{t+1} pour lesquels la transition est impossible ($a_{S_*^t, S_*^{t+1}} = 0$). Dans ce cas, la probabilité globale de la séquence d'états résultante est strictement nulle.

Les algorithmes Forward et Viterbi partagent la même topologie de programmation dynamique sur le treillis. Ils correspondent à une instanciation formelle du même algorithme abstrait sur deux *semi-anneaux* distincts $(K, \oplus, \otimes, \bar{0}, \bar{1})$:

Caractéristique	**Semi-anneau des Réels (Forward)**	**Semi-anneau Tropical (Viterbi)**
Support K	$\mathbb{R}_+$	$\mathbb{R}_+$
Élément nul $\bar{0}$	0	0
Élément unité $\bar{1}$	1	1
Addition $\oplus$	$+$ (somme des alternatives)	max (sélection du meilleur chemin)
Multiplication $\otimes$	$\times$ (accumulation causale)	$\times$ (accumulation causale)

TABLE 4.1 – Correspondance algébrique entre Forward et Viterbi via les semi-anneaux.

4.3.3 Stabilité Numérique et Domaine Logarithmique

Sur de longues séquences, la multiplication itérative de probabilités inférieures à 1 entraîne rapidement un dépassement de capacité par le bas (*arithmetic underflow*) sur les flottants standard. Pour éviter l'écrasement à zéro, on manipule les logarithmes des probabilités. Pour deux log-probabilités $u = \ln p$ et $v = \ln q$:

- La multiplication devient une addition : $\ln(p \times q) = u + v$.
- Dans l'algorithme de Viterbi, la monotonie du logarithme commute avec le maximum :

$$\ln \delta_t(j) = \ln b_j(O_t) + \max_{1 \le i \le N} \big[\ln \delta_{t-1}(i) + \ln a_{ij}\big]. \tag{4.3.22}$$

Pour l'algorithme Forward, l'addition de probabilités dans le domaine log nécessite le calcul de $\ln(p+q) = \ln(\mathrm{e}^u + \mathrm{e}^v)$. De manière générale, pour un vecteur $l = (l_1, \ldots, l_n) \in \mathbb{R}^n$, on définit l'opérateur *logsumexp* :

$$\mathrm{LSE}(l) \triangleq \ln\left(\sum_{i=1}^{n} \mathrm{e}^{l_i}\right). \tag{4.3.23}$$

Si certains composants de l sont très grands ($l_i \gg 0$), le calcul brut de e^{l_i} provoque un dépassement par le haut (*overflow*).

Proposition (Factorisation stable de Log-Sum-Exp) Pour tout scalaire $a \in \mathbb{R}$:

$$\mathrm{LSE}(l) = a + \mathrm{LSE}(l-a) = a + \ln\left(\sum_{i=1}^{n} \mathrm{e}^{l_i - a}\right). \tag{4.3.24}$$

Démonstration En factorisant par e^a dans la somme :

$$\begin{aligned}
\mathrm{LSE}(l) &= \ln\left(\sum_{i=1}^{n} \mathrm{e}^{l_i}\right) = \ln\left(\sum_{i=1}^{n} \mathrm{e}^{a} \cdot \mathrm{e}^{l_i - a}\right) \\
&= \ln\left(\mathrm{e}^{a} \sum_{i=1}^{n} \mathrm{e}^{l_i - a}\right) = \ln(\mathrm{e}^a) + \ln\left(\sum_{i=1}^{n} \mathrm{e}^{l_i - a}\right) \\
&= a + \ln\left(\sum_{i=1}^{n} \mathrm{e}^{l_i - a}\right).
\end{aligned} \tag{4.3.25}$$

En pratique, on pose $a = \max_{1 \le i \le n} l_i$. Ainsi, pour tout i, $l_i - a \le 0$, ce qui borne $\mathrm{e}^{l_i - a} \in]0, 1]$ et neutralise tout risque d'overflow.

4.3.4 Problème 3 : Apprentissage des Paramètres

L'objectif de l'apprentissage est d'estimer les paramètres du modèle $\lambda = (\pi, A, B)$ à partir d'un ensemble de séquences d'entraînement.

Proposition (Cas Supervisé) Lorsque les séquences d'observations et les séquences d'états associées sont intégralement observées (par exemple en étiquetage POS annoté manuellement), l'estimation des paramètres par *Maximum de Vraisemblance* (MLE) se réduit à des ratios de fréquences empiriques :

$$\hat{\pi}_i = \frac{\mathrm{Count}(S_1 = s_i)}{\sum_{j=1}^{N} \mathrm{Count}(S_1 = s_j)}, \tag{4.3.26}$$

$$\hat{a}_{ij} = \frac{\mathrm{Count}(S_{t-1} = s_i, S_t = s_j)}{\sum_{k=1}^{N} \mathrm{Count}(S_{t-1} = s_i, S_t = s_k)}, \tag{4.3.27}$$

$$\hat{b}_j(k) = \frac{\mathrm{Count}(S_t = s_j, O_t = v_k)}{\sum_{m=1}^{M} \mathrm{Count}(S_t = s_j, O_t = v_m)}. \tag{4.3.28}$$

Proposition (Cas Non-Supervisé : Algorithme EM) Lorsque les états latents ne sont pas observés, l'optimisation directe de la log-vraisemblance marginale $\ln \mathbb{P}(O \mid \lambda) = \ln \sum_S \mathbb{P}(O, S \mid \lambda)$ est non convexe. On applique l'algorithme d'*Espérance-Maximisation* (EM), qui prend le nom d'algorithme de *Baum-Welch* dans le cadre des HMM.
L'algorithme itère deux étapes jusqu'à convergence :

1. **Étape E (Espérance)** : À l'aide des paramètres courants $\lambda^{(k)}$ et via les algorithmes Forward et Backward, on calcule les distributions a posteriori sur les variables latentes :

- Probabilité d'être dans l'état s_i au temps t :

$$\gamma_t(i) = \mathbb{P}(S_t = s_i \mid O, \lambda^{(k)}) = \frac{\alpha_t(i)\,\beta_t(i)}{\sum_{j=1}^{N} \alpha_t(j)\,\beta_t(j)}. \tag{4.3.29}$$

- Probabilité de transiter de s_i à s_j entre les instants t et $t+1$:

$$\xi_t(i,j) = \mathbb{P}(S_t = s_i, S_{t+1} = s_j \mid O, \lambda^{(k)}) = \frac{\alpha_t(i)\,a_{ij}\,b_j(O_{t+1})\,\beta_{t+1}(j)}{\mathbb{P}(O \mid \lambda^{(k)})}. \tag{4.3.30}$$

2. **Étape M (Maximisation)** : On met à jour les paramètres $\lambda^{(k+1)}$ en maximisant l'espérance de la log-vraisemblance des données complètes :

$$\pi_i^{(k+1)} = \gamma_1(i), \tag{4.3.31}$$

$$a_{ij}^{(k+1)} = \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{t=1}^{T-1} \gamma_t(i)}, \tag{4.3.32}$$

$$b_j(m)^{(k+1)} = \frac{\sum_{t=1}^{T} \gamma_t(j) \cdot \mathbb{I}(O_t = v_m)}{\sum_{t=1}^{T} \gamma_t(j)}. \tag{4.3.33}$$

L'algorithme garantit la croissance monotone de la vraisemblance marginale $\mathbb{P}(O \mid \lambda)$ à chaque itération et converge vers un extremum local du paysage d'optimisation.

Chapitre 5

Apprentissage par Renforcement

Contents

5.1 Cadre général 29
5.1.1 Agent et environnement 29
5.1.2 Cadre mathématique : processus Markovien 30
5.1.3 Politique 31
5.2 Des équations de Bellman à la Programmation Dynamique 32
5.2.1 Premier algorithme : rétropropagation 32
5.2.2 Fonction Q 33
5.2.3 Amélioration itérative de politique (*Policy Iteration*) 34
5.2.4 Itération sur la valeur (*Value Iteration*) 34
5.3 Apprentissage par renforcement 35
5.3.1 Cadre général sans modèle - *Model-Free* 35
5.3.2 Algorithme de Q-learning 35
5.3.3 Alternative *On-Policy* : L'algorithme SARSA 36
5.3.4 Compromis Exploration vs Exploitation 37

Dans ce chapitre, nous présentons et développons brièvement la théorie de l'Apprentissage par Renforcement. Au contraire de l'apprentissage dit *supervisé* traité jusqu'ici, on ne dispose plus de *données* ou *d'étiquettes*, mais d'un *agent* dans un *environnement*.

5.1 Cadre général

Commençons par formaliser le cadre général de *l'apprentissage par renforcement*.

5.1.1 Agent et environnement

On considère donc un *environnement* dans lequel évolue un *agent* (voir Figure 5.1). Ce dernier peut effectuer des *actions* qui influent sur son état actuel ainsi que celui de l'environnement. On pourra classifier/ordonner les différentes actions jouées par l'agent en fonction de leur *intérêt* à réaliser une tâche : pour cela, on leur associera un coût et une pénalité.
L'objectif est de faire *apprendre* à l'agent un comportement en fonction de la structure de l'environnement pour qu'il réalise une tâche donnée. On appellera cela l'apprentissage d'une *politique*.

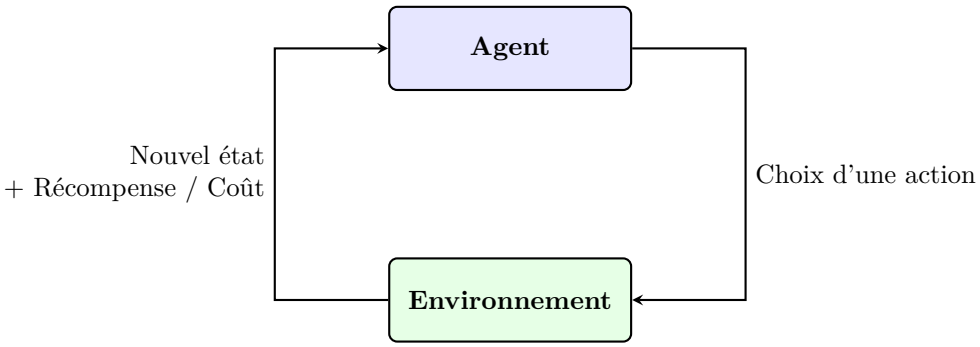


FIGURE 5.1 – Schéma simplifié de l'évolution d'un agent dans un environnement.

Exemple (Gestion de l'énergie d'un satellite en orbite) Soit un nano-satellite en orbite basse équipé d'une batterie et d'un instrument scientifique (caméra ou émetteur). À chaque révolution orbitale, un agent embarqué doit décider du mode d'opération en fonction de la réserve d'énergie disponible. On discrétise la capacité de la batterie en trois niveaux de charge :

- s_0 : *Batterie critique* (niveau minimal de survie),
- s_1 : *Batterie moyenne*,
- s_2 : *Batterie pleine*.

Depuis chaque état, l'agent peut choisir entre deux modes d'action :

- a_{eco} (*Mode veille / recharge*) : les instruments sont coupés pour recharger la batterie grâce aux panneaux solaires,
- a_{mission} (*Exécution de mission*) : l'instrument est activé pour acquérir et transmettre des données scientifiques, ce qui consomme une quantité importante d'énergie.

Chaque transmission réussie rapporte une récompense positive (+10), tandis que laisser le satellite tomber en décharge critique (s_0) ou tenter d'activer la charge utile sans énergie suffisante engendre une pénalité sévère (coût négatif ou risque de perte de la mission).

5.1.2 Cadre mathématique : processus Markovien

Définissons maintenant un cadre mathématique à l'étude des modèles d'apprentissage par renforcement. Dans ce cours, on fait l'hypothèse que chaque action faite par l'agent ne dépend que de son état courant et de la dernière action effectuée. On appelle cela une hypothèse *markovienne* : le cadre défini ci-dessous dépend de celle-ci.

Définition (**Cadre Markovien**) . On peut modéliser ce problème d'apprentissage de la façon suivante :

- S représente l'espace des états,
- A est l'ensemble des actions,
- $A(s)$ est l'ensemble des actions réalisables dans l'état $s \in S$,
- $r(s, a) \in \mathbb{R}$ est la récompense immédiate lorsque l'agent exécute l'action $a \in A(s)$ à partir de l'état $s \in S$,
- $\delta(s, a) \in S$ donne l'état résultant de l'action a depuis l'état s.

Remarque On suppose ici que l'on a une parfaite connaissance de l'environnement (i.e espace d'états S) ainsi que des actions que l'on peut effectuer dans un état (fonction A). D'autre part, dans des problèmes d'apprentissage par renforcement complexes, l'application r n'est

pas toujours connue : on n'est pas capable de prévoir exactement quelles vont être les récompenses/pénalités obtenus en effectuant une action $a \in A(s)$ depuis un état $s \in S$.

Exemple (Gestion de l'énergie d'un satellite en orbite) On peut désormais reprendre l'exemple introduit précédemment pour lui appliquer le formalisme tout juste défini. On a donc :

- Espace des états : $S = \{s_0, s_1, s_2\}$ avec $|S| = 3$.
- Espace des actions : $A = \{a_{\text{eco}}, a_{\text{mission}}\}$, avec $A(s) = A$ pour tout $s \in S$.
- Dynamique déterministe $\delta : S \times A \to S$:

$$\begin{aligned} &\delta(s_0, a_{\text{eco}}) = s_1, && \delta(s_0, a_{\text{mission}}) = s_0, \\ &\delta(s_1, a_{\text{eco}}) = s_2, && \delta(s_1, a_{\text{mission}}) = s_0, \\ &\delta(s_2, a_{\text{eco}}) = s_2, && \delta(s_2, a_{\text{mission}}) = s_1. \end{aligned}$$

- Fonction de récompense $r : S \times A \to \mathbb{R}$:

$$\begin{aligned} &r(s_0, a_{\text{eco}}) = 0, && r(s_0, a_{\text{mission}}) = -10 \quad \text{(échec / pénalité critique)}, \\ &r(s_1, a_{\text{eco}}) = 0, && r(s_1, a_{\text{mission}}) = +10, \\ &r(s_2, a_{\text{eco}}) = 0, && r(s_2, a_{\text{mission}}) = +10. \end{aligned}$$

Dans certaines situations, on peut définir des modèles *non déterministes*. Autrement dit, il se peut que les transitions entre les états soient probabilistes : pour un même état de départ et une même action, l'agent peut se retrouver dans des états différents.

5.1.3 Politique

Dans le cadre de l'apprentissage par renforcement, la notion d'apprentissage est modélisée par une *politique*.

Définition (**Politique**) . Soit le cadre mathématique défini plus haut. On appelle *politique* associée à un agent et un environnement donné, l'application $\pi : S \longrightarrow A$ telle que $\forall s \in S, \pi(s) \in A(s)$.

Cette application définit l'action que doit effectuer l'agent pour chaque état atteignable $s \in S$. La propriété $\forall s \in S, \pi(s) \in A(s)$ permet de toujours orienter l'agent vers un état atteignable depuis l'état courant.

Remarque Une politique est une application $\pi : S \longrightarrow A$, il existe donc $|A|^{|S|}$ politiques possible pour chaque ensemble A et S. Dans notre exemple précédent il existe donc $2^3 = 8$ politiques différentes. On pourrait ainsi toutes les considérer et choisir celle qui maximise la récompense r pour l'ensemble des transitions. En pratique, il est souvent impossible d'énumérer toutes les politiques possibles pour choisir la meilleure. On doit donc développer des approches algorithmiques plus performantes.

Définition (**Récompense à long terme**) . L'objectif de l'apprentissage d'une politique est de maximiser les récompenses de l'agent sur le long terme. On définit donc la *récompense à long terme* comme la somme des récompenses obtenues depuis le début de la simulation :

$$R(s) = \sum_{i=0}^{\infty} \gamma^i \cdot r(s_i, a_i) \tag{5.1.1}$$

pour une séquence d'états et d'actions $s = (s_0, a_0, s_1, a_1, \dots)$ donnée. On prendra $\gamma \in]0, 1]$

tel que $\gamma < 1$ favorisant ainsi les récompenses immédiates et la convergence de la somme si les récompenses sont bornées.

Proposition Soit π une politique, on peut alors calculer la récompense obtenue par l'application de cette politique à partir de l'état $s_0 \in S$ avec :

$$V_\pi(s_0) = R(\pi, s_0) = \sum_{i=0}^{\infty} \gamma^i \cdot r(s_i, a_i) \quad \text{avec} \quad a_i = \pi(s_i), s_{i+1} = \delta(s_i, a_i). \tag{5.1.2}$$

Proposition (Comparaison de politiques) On peut comparer deux politiques π et π' sur deux mêmes espaces A et S en comparant simplement leur récompense à long terme. On dira alors que $\pi \succeq \pi'$ ssi

$$\forall s_0 \in S, \quad R(\pi, s_0) \geqslant R(\pi', s_0). \tag{5.1.3}$$

On peut alors se demander s'il existe une politique optimale pour un problème d'apprentissage par renforcement donné et s'il est possible de concevoir un algorithme qui nous permette de l'apprendre.

5.2 Des équations de Bellman à la Programmation Dynamique

Dans cette section, on développe différentes approches et algorithmes pour l'apprentissage d'une politique. On se base pour cela sur les équations de Bellman qui utilisent l'hypothèse Markovienne.

5.2.1 Premier algorithme : rétropropagation

Proposition (Équations de Bellman) Soient S et A respectivement un espace d'état et un espace d'actions. On note r la récompense, δ les transitions et γ le facteur de rabais de ce problème d'apprentissage par renforcement. En adoptant l'hypothèse Markovienne sur la dépendances des états entre eux, pour tout état $s \in S$ et politique π, on a :

$$R(\pi, s) = r(s, \pi(s)) + \gamma R(\pi, \delta(s, \pi(s))). \tag{5.2.1}$$

À partir de cette proposition, on est donc capable de calculer une politique optimale si l'on connaît toutes les transitions δ et les récompenses r. Il suffit alors de résoudre un système linéaire où les inconnues sont $R(\pi, s)$.
L'algorithme ci-dessous utilise la proposition précédente pour calculer itérativement la récompense à long terme d'une politique.

Algorithm 4 Calcul de la récompense à long terme

1. $R(\pi, s) \longleftarrow 0$ pour tout $s \in S$.
2. Tant que pas fini, faire :
   (a) Pour tout $s \in S$, si $a = \pi(s), s' = \delta(s, a)$ alors :

   $$temp(s) \longleftarrow r(s, a) + \gamma R(\pi, s')$$

   (b) $R(\pi, s) \longleftarrow temp(s)$ pour tout $s \in S$.

À noter qu'il est toujours nécessaire de connaître parfaitement les transition δ et les récompenses pour chaque transition possible r.

Proposition (Amélioration locale d'une politique) On peut alors améliorer localement une politique en comparant les récompenses de chaque transition possibles depuis un état. Soit π une politique donnée, $s \in S$ et $a \in A(s)$. On pose $s' = \delta(s, a) \in S$, si :

$$R(\pi, s) < r(s, a) + \gamma R(\pi, s') \tag{5.2.2}$$

alors on pose $\pi'(x) = \pi(x)$ pour tout état $x \in S$ sauf s où $\pi'(s) = a$. On aura alors construit une politique π' meilleure que π (i.e $\pi' \succ \pi$).

5.2.2 Fonction Q

Les propositions précédentes basées sur la récompense à long terme $R(\cdot, s)$ depuis un état $s \in S$ semblent montrer que cette application est la clé pour calculer (ou estimer) une politique optimale pour notre agent. On définit donc une fonction permettant d'évaluer la récompense à long terme d'une politique π depuis un état $s \in S$ en commençant par effectuer une action $a \in A(s)$. Cette fonction, notée Q, sera directement définie par l'équation 5.2.1.

Définition (**Fonction** Q) . Soient r, δ, γ défini dans le contexte précédent et π une politique. On défini la fonction Q par :

$$\forall s \in S, \forall a \in A(s), \quad Q(\pi, s, a) = r(s, a) + \gamma R(\pi, \delta(s, a)). \tag{5.2.3}$$

La propriété suivante est obtenue par un jeu d'écriture sur l'équation 5.2.1 et la défintion de Q.

Propriété (**Évaluation d'un état**) . D'après la définition précédente, on peut remarquer que :

$$\forall s \in S, \forall a \in A(s), \quad R(\pi, s) = Q(\pi, s, \pi(s)). \tag{5.2.4}$$

Ainsi, pour une politique donnée, Q vérifie la relation suivante :

$$\boxed{Q(\pi, s, a) = r(s, a) + \gamma Q(\pi, s', \pi(s'))} \tag{5.2.5}$$

où $s' = \delta(s, a)$.

L'évaluation via la fonction Q permettrait donc d'analyser l'impact d'une déviation ponctuelle par rapport à la politique courante π. $Q(\pi, s, a)$ mesure le *gain espéré* si l'agent effectue immédiatement l'action a, puis se conforme à π pour toutes les étapes futures.

Théorème (**Caractérisation de Bellman**) . Une politique π^* est *optimale* si et seulement si, pour tout état $s \in S$ et pour toute action réalisable $a \in A(s)$, la fonction Q associée vérifie le système d'équations non linéaires :

$$Q(\pi^*, s, a) = r(s, a) + \gamma \max_{a' \in A(s')} Q(\pi^*, s', a') \tag{5.2.6}$$

où $s' = \delta(s, a)$. La politique optimale déterministe s'en déduit alors de manière gloutonne :

$$\pi^*(s) = \arg\max_{a \in A(s)} Q(\pi^*, s, a), \quad \forall s \in S. \tag{5.2.7}$$

Dans le cas simple de l'Exemple 2, il était possible de calculer explicitement la politique optimale par force brute en considérant les 8 politiques possibles. Or, lorsque ce nombre devient exponentiel, la complexité algorithmique de résolution rend la recherche infaisable. La solution consiste donc à résoudre le système présenté en 5.2.7. Toutefois, celui-ci comporte un opérateur max à chaque ligne : nos algorithmes itératifs d'inversion de matrice ne sont donc plus applicables.
C'est pourquoi nous avons recours à la *programmation dynamique*.

5.2.3 Amélioration itérative de politique (*Policy Iteration*)

Nous présentons un premier algorithme de *policy-iteration* permettant d'améliorer une politique de façon itérative basé sur la Proposition 15. L'idée est de considérer toutes les actions possibles $A(s)$ pour un état $s \in S$ donné et de calculer la récompense à long terme de chacune de ces actions $a \in A(s)$. On choisit ensuite celle avec la récompense la plus importante pour modifier notre politique.
L'algorithme d'*itération sur les politiques* alterne ainsi deux étapes fondamentales jusqu'à convergence. La première est *l'évaluation de la politique courante* par le calcul exact ou approché de $R(\pi, \cdot)$ ou $Q(\pi, \cdot, \cdot)$. La seconde est *l'amélioration gloutonne* : il sélectionne l'action qui maximise la fonction Q courante en chaque état.

Algorithm 5 Amélioration itérative de politique (*Policy Iteration*)

1. Initialiser arbitrairement une politique π_0 (par exemple uniforme ou aléatoire).
2. **Tant que** $\pi_{k+1} \neq \pi_k$ (ou condition d'arrêt satisfaite), **faire** :
   (a) **Évaluation** : pour tout $s \in S$ et tout $a \in A(s)$, calculer $Q(\pi_k, s, a)$:
   $$Q(\pi_k, s, a) = r(s, a) + \gamma R(\pi_k, \delta(s, a)). \tag{5.2.8}$$
   (b) **Amélioration** : pour tout état $s \in S$, mettre à jour la politique de manière gloutonne :
   $$\pi_{k+1}(s) \longleftarrow \arg\max_{a \in A(s)} Q(\pi_k, s, a). \tag{5.2.9}$$
3. Retourner la politique optimale $\pi^* = \pi_k$.

Remarque (Convergence et coût de calcul) Puisque le nombre de politiques déterministes est fini ($|A|^{|S|}$) et que chaque étape d'amélioration garantit $\pi_{k+1} \succeq \pi_k$, l'algorithme converge en un nombre fini d'itérations vers une politique optimale globale. Cependant, évaluer intégralement la fonction $Q(\pi_k, \cdot, \cdot)$ à chaque boucle exige de résoudre un système de $|S|$ équations linéaires, ce qui devient prohibitif quand $|S|$ est grand.

5.2.4 Itération sur la valeur (*Value Iteration*)

Algorithm 6 Itération sur la valeur (*Value Iteration*)

1. Initialiser $R(s) \longleftarrow 0$ pour tout $s \in S$.
2. **Tant que** non convergé (par exemple $\max_s |R_{\text{nouveau}}(s) - R(s)| > \varepsilon$), **faire** :
   (a) Pour tout $s \in S$:
   $$\forall a \in A(s), \quad Q(s, a) \longleftarrow r(s, a) + \gamma R(\delta(s, a)), \tag{5.2.10}$$
   $$R_{\text{temp}}(s) \longleftarrow \max_{a \in A(s)} Q(s, a). \tag{5.2.11}$$
   (b) $R(s) \longleftarrow R_{\text{temp}}(s)$ pour tout $s \in S$.
3. **Extraction de la politique finale** : pour tout $s \in S$,
   $$\pi^*(s) \longleftarrow \arg\max_{a \in A(s)} \left(r(s, a) + \gamma R(\delta(s, a))\right). \tag{5.2.12}$$

Pour contourner le coût de l'évaluation complète d'une politique intermédiaire, l'*itération sur la valeur* fusionne les étapes d'évaluation et d'amélioration en une seule mise à jour basée sur l'opérateur de Bellman optimal.
Au lieu de calculer le résultat de la politique courante π_k jusqu'à la fin de la simulation (Étape d'évaluation) qu'on modifie un de ses termes (Étape d'amélioration) puis que l'on recalcule le résultat de cette nouvelle politique π_{k+1}, on choisit plutôt de calculer la récompense à long terme pour chaque action $a \in A(s)$ puis on choisit la meilleure. On effectue ces deux étapes tant que l'on n'a pas convergé. La politique est obtenue en choisissant à chaque étape la meilleure action au sens des récompenses.

Remarque (Q-learning) Ces deux algorithmes de programmation dynamique supposent la connaissance exacte du modèle de l'environnement, à savoir la fonction de transition δ et la fonction de récompense r. Lorsque ces fonctions sont inconnues de l'agent, celui-ci doit interagir directement avec l'environnement pour échantillonner ses transitions : c'est le cadre du *Q-learning*.

5.3 Apprentissage par renforcement

La plupart du temps on ne dispose pas de modèle explicite ou calculable de l'environnement (fonctions r et δ) : les algorithmes précédents ne fonctionnent plus. On va donc chercher à les estimer par *échantillonnage* en explorant l'environnement et en observant les récompenses obtenues pour chaque couple $(s, a) \in S \times A(s)$.

5.3.1 Cadre général sans modèle - *Model-Free*

Lorsque l'agent ne connaît pas les lois r et δ, il ne peut plus évaluer l'opérateur de Bellman par calcul direct. Il doit interagir avec l'environnement par essais et erreurs en collectant des trajectoires sous forme d'expériences :

$$(s_t, a_t, r_{t+1}, s_{t+1}) \in S \times A \times \mathbb{R} \times S. \tag{5.3.1}$$

Algorithm 7 Schéma générique d'interaction par épisodes

1. Initialiser une représentation du MDP (typiquement une table de valeurs $Q(s, a)$ arbitraire).
2. **Répéter** pour chaque épisode :
   (a) Sélectionner un état initial $s_0 \in S$.
   (b) **Tant que** l'état courant s n'est pas terminal, **faire** :
      i. Choisir une action $a \in A(s)$ selon une stratégie d'exploration.
      ii. Exécuter a, puis observer la récompense immédiate r et l'état résultant s'.
      iii. Mettre à jour la fonction $Q(s, a)$ à partir du quadruplet (s, a, r, s').
      iv. Transitionner : $s \longleftarrow s'$.

5.3.2 Algorithme de Q-learning

Introduit par Watkins (1989), le Q-learning cherche à approximer la fonction d'action-valeur optimale Q^* sans jamais modéliser la dynamique δ ni la fonction r. Dans un cadre déterministe, chaque transition observée (s, a, r, s') lors d'une trajectoire permet d'effectuer une mise à jour par écrasement direct inspirée de l'équation d'optimalité de Bellman :

$$Q(s, a) \longleftarrow r + \gamma \max_{a' \in A(s')} Q(s', a'). \tag{5.3.2}$$

Théorème (**Convergence en univers déterministe**) . Si les récompenses immédiates $r(s,a)$ sont bornées, si $\gamma \in [0,1[$, et si la stratégie de sélection d'actions garantit que chaque couple (s,a) est visité une infinité de fois, alors la table $Q(s,a)$ converge vers la fonction optimale $Q^*(s,a)$.

Remarque (Accélération par rétropropagation intra-épisode) Lorsque les récompenses sont rares (*sparse rewards*) — par exemple si la seule récompense non nulle n'intervient qu'à l'état final s_T —, mettre à jour la table au fil de l'eau propage l'information d'un seul pas par épisode.
Pour accélérer la diffusion de l'information, on stocke la trajectoire complète $(s_0, a_0, r_1, s_1, \ldots, s_T)$ et l'on applique les mises à jour en sens inverse, de $t = T-1$ jusqu'à $t = 0$:

$$Q(s_t, a_t) \longleftarrow r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'). \tag{5.3.3}$$

La récompense finale rétropropage immédiatement sa contribution sur l'ensemble des états visités au cours de l'épisode.

Proposition (Cas non-déterministe) Dans les systèmes stochastiques, une même action a exécutée dans un même état s peut conduire à des états d'arrivée s' distincts avec des récompenses différentes. Réécraser brutalement la valeur $Q(s,a)$ provoquerait des oscillations permanentes sans convergence.
On stabilise donc la dynamique en introduisant un taux d'apprentissage $\alpha \in]0,1]$, calculant ainsi une moyenne glissante temporelle entre l'estimation passée et la nouvelle observation :

$$Q(s,a) \longleftarrow (1-\alpha)Q(s,a) + \alpha \left(r + \gamma \max_{a' \in A(s')} Q(s', a') \right). \tag{5.3.4}$$

Remarque (*TD-error*) Cette équation de mise à jour se reformule de façon canonique sous la forme d'un ajustement proportionnel à un résidu d'erreur :

$$Q(s,a) \longleftarrow Q(s,a) + \alpha \cdot \delta_{\text{TD}} \tag{5.3.5}$$

où le terme δ_{TD} représente *l'erreur de différence temporelle* (erreur TD) :

$$\delta_{\text{TD}} \triangleq \underbrace{\left(r + \gamma \max_{a' \in A(s')} Q(s', a') \right)}_{\text{Cible TD (valeur visée)}} - \underbrace{Q(s,a)}_{\text{Estimation courante}} . \tag{5.3.6}$$

Remarque (Régime du taux d'apprentissage) La convergence théorique vers Q^* en environnement stochastique impose les conditions de Robbins-Monro sur la suite des pas (α_t) :

$$\sum_{t=1}^{\infty} \alpha_t(s,a) = \infty \quad \text{et} \quad \sum_{t=1}^{\infty} \alpha_t(s,a)^2 < \infty. \tag{5.3.7}$$

En pratique, on adopte souvent soit un pas constant faible ($\alpha \in [0{,}01; 0{,}1]$), soit une décroissance dépendante du nombre de passages : $\alpha(s,a) = \frac{1}{1+\text{visites}(s,a)}$.

5.3.3 Alternative *On-Policy* : L'algorithme SARSA

Une distinction fondamentale structure les algorithmes d'apprentissage par renforcement :

- **Approche hors politique (*Off-Policy*)** : l'algorithme apprend la valeur d'une politique cible tout en suivant une politique de comportement différente. Le Q-learning est *off-policy* car sa cible de mise à jour utilise l'opérateur optimal $\max_{a'}$, indépendamment de l'action qui sera réellement choisie à l'instant suivant.

- **Approche sur politique (*On-Policy*)** : l'algorithme évalue et met à jour la valeur de la politique effectivement suivie par l'agent.

L'algorithme SARSA (*State-Action-Reward-State-Action*) applique ce principe : au lieu d'optimiser par un max théorique, il utilise l'action $a' = \pi(s')$ réellement sélectionnée pour l'étape suivante :

$$Q(s,a) \longleftarrow (1-\alpha)Q(s,a) + \alpha\Big(r + \gamma Q(s',a')\Big) \quad \text{avec } a' \sim \pi(\cdot \mid s'). \tag{5.3.8}$$

5.3.4 Compromis Exploration vs Exploitation

Dans le cadre d'un apprentissage *model-free*, l'agent est soumis à un arbitrage permanent entre *exploration* et *exploitation* pour d'une part, comprendre l'environnement dans lequel il évolue mais aussi construire rapidement une politique optimale d'exploitation de cet environnement. On parlera de compromis *exploration/exploitation* :

- *Exploiter* : sélectionner l'action maximisant $Q(s,a)$ pour maximiser les gains connus.
- *Explorer* : essayer des actions d'apparence sous-optimale pour découvrir de meilleures trajectoires à long terme.

Définition (**Stratégies de sélection d'actions**) . Soit un état $s \in S$. On distingue principalement trois mécanismes de sélection :

1. **Politique gloutonne (*Greedy*)** : aucun compromis d'exploration, choix systématique du meilleur score :
$$a = \arg\max_{a \in A(s)} Q(s,a). \tag{5.3.9}$$

2. **Politique ε-gloutonne (ε-Greedy)** : avec une probabilité $1-\varepsilon$, l'agent exploite la meilleure action ; avec une probabilité ε, il tire une action au hasard selon une loi uniforme sur $A(s)$:
$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{|A(s)|} & \text{si } a = \arg\max_{a'} Q(s,a'), \\ \dfrac{\varepsilon}{|A(s)|} & \text{sinon.} \end{cases} \tag{5.3.10}$$

3. **Sélection Softmax (loi de Boltzmann)** : chaque action reçoit une probabilité de tirage pondérée par sa valeur relative, modulée par un paramètre de température $\tau > 0$:
$$\pi(a \mid s) = \frac{\exp\left(\dfrac{Q(s,a)}{\tau}\right)}{\sum_{a' \in A(s)} \exp\left(\dfrac{Q(s,a')}{\tau}\right)}. \tag{5.3.11}$$
Lorsque $\tau \to \infty$, la sélection tend vers une loi uniforme (exploration pure) ; lorsque $\tau \to 0^+$, elle converge vers la politique gloutonne déterministe (exploitation pure).

Remarque La politique ε-gloutonne peut être intéressante pour régler le compromis exploration/exploitation. En début d'apprentissage, ε peut être relativement haut pour favoriser l'exploration de l'environnement. Au fur et à mesure des épisodes, ε peut diminuer pour favoriser progressivement l'exploitation de l'environnement découvert.